LABORATORY OBSERVATION

Course:	Full Stack Web Development	**Experiment No.:**	6 (Exp 2)
Course Code:	23CM4121	**Date:**	05-10-2026
Student Name:	Nimidithalli Nandu	**Roll No.:**	A24126552100

1. TITLE

Working with Node.js Built-in Core Modules (OS, Path, and FS)

2. AIM

To understand and implement core Node.js runtime modules including os for hardware and environment queries, path for cross-platform path resolution, and fs for synchronous file I/O operations.

3. OBJECTIVE

- Query host operating system platform, CPU architecture, home directory, and core count.
- Manipulate and parse filesystem path strings using the path module.
- Create, write, and read files synchronously using the fs module.
- Verify host machine diagnostics and file system integrity.

4. THEORY

Node.js contains built-in core modules accessible without installing external packages via require().

os Module: Exposes operating system metrics including platform, architecture, CPU count, and user home path.

path Module: Provides utilities for resolving and parsing file system paths in an OS-independent manner using path.join(), path.basename(), and path.extname().

fs Module: Enables synchronous and asynchronous interaction with the host file system. writeFileSync() creates and overwrites file data, while readFileSync() retrieves file content.

5. ALGORITHM / PROCEDURE

1. Require the built-in os module and log os.platform(), os.arch(), os.homedir(), and os.cpus().length.

2. Require the path module and construct normalized path students/data.txt using path.join().

3. Extract and log the file name using path.basename() and extension using path.extname().

4. Require the fs module and write "Hello from Node.js!" into message.txt using writeFileSync().

5. Read the content of message.txt synchronously using readFileSync() with utf8 encoding and print to console.

6. PROGRAM

code.js

```
// os module
const os = require("os");

console.log("Operating System:", os.platform());
console.log("CPU Architecture:", os.arch());
console.log("Home Directory:", os.homedir());
console.log("Number of CPUs:", os.cpus().length);

// path module
const path = require("path");

const filePath = path.join("students", "data.txt");

console.log("File Path:", filePath);
console.log("File Name:", path.basename(filePath));
console.log("Extension:", path.extname(filePath));

// fs module
const fs = require("fs");

fs.writeFileSync("message.txt", "Hello from Node.js!");

const data = fs.readFileSync("message.txt", "utf8");

console.log("File Content:", data);
```

7. CODE EXPLANATION

Code Element	Detailed Explanation
const os = require("os")	Imports the core OS module for querying hardware and platform properties.
os.platform() / os.arch()	Retrieves host operating system identifier (win32) and CPU architecture (x64).
path.join("students", "data.txt")	Constructs a platform-appropriate relative file path.
fs.writeFileSync(...)	Synchronously creates message.txt with the provided string content.
fs.readFileSync(...)	Synchronously reads back the file text in UTF-8 format.

8. TEST CASES AND EXECUTION DATA

Test Case	Action / Input	Expected Result	Actual Result	Status
TC-01	Query OS platform and arch	Return win32 and x64	Platform win32 and x64 displayed	**Pass**
TC-02	Query CPU count and homedir	Display CPU cores and user directory	Logged 12 CPUs and home directory	**Pass**
TC-03	Path joining and parsing	Output normalized path and extension	File path and .txt extension logged	**Pass**
TC-04	File write and read	Create message.txt and retrieve content	File content "Hello from Node.js!" logged	**Pass**

9. OUTPUT

Terminal Execution of nodejs_modules/code.js

```
PS C:\Users\sharat\OneDrive\Desktop\fs observation tasks\task6\nodejs_modules> node code.js
Operating System: win32
CPU Architecture: x64
Home Directory: C:\Users\sharat
Number of CPUs: 12
File Path: students\data.txt
File Name: data.txt
Extension: .txt
File Content: Hello from Node.js!
```

10. RESULT

Built-in Node.js modules (os, path, fs) were successfully implemented to inspect system specifications, manipulate filesystem paths, and perform synchronous file operations.